

Pipes

Part II

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

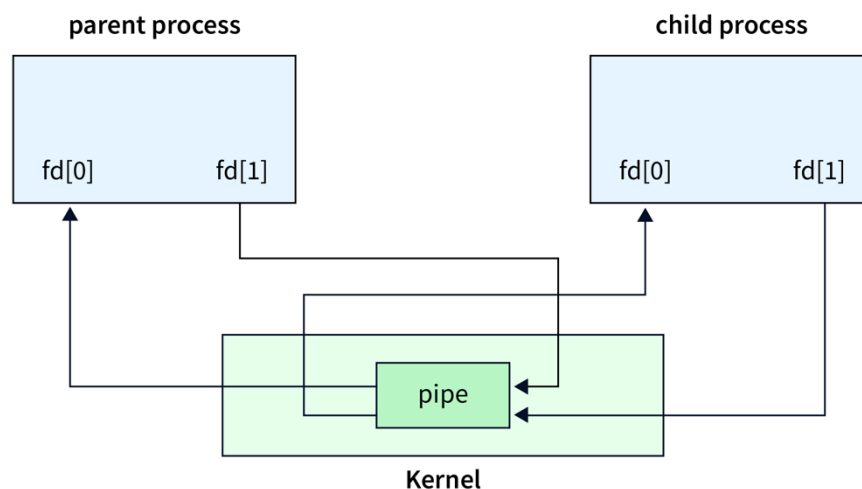
Luís Nogueira (lmn@isep.ipp.pt)

Recall from previous class

- A pipe is a communication channel that connects two processes and allows for **unidirectional** data flow
- From the programmer's perspective, the interface is the same as for files
 - A pipe is represented by an array of descriptors with two positions (0 – reading entry; 1 – write entry)
 - The flow of bytes written to the write end of the pipe is read from the read end
- Contrary to files, **reading removes** the data from the pipe, and **synchronization** of read/write **is guaranteed**

Recall from previous class

- After `fork()`, parent and child have the same descriptors



- **Unused descriptors must be closed**

- EOF is only returned in a `read()` if all write descriptors are closed
- If all read descriptors are closed, `write()` generates `SIGPIPE` (and fails with `EPIPE` if the signal is ignored)

Exercise 1

- **Given two arrays of 1000 position each, implement a program that creates 5 child processes to calculate the sum of the arrays**
 - Each child sums 200 positions of the array and send this partial sum to the parent process using a pipe
 - The parent gets all partial sums from the pipe, computes the total sum, and prints the result

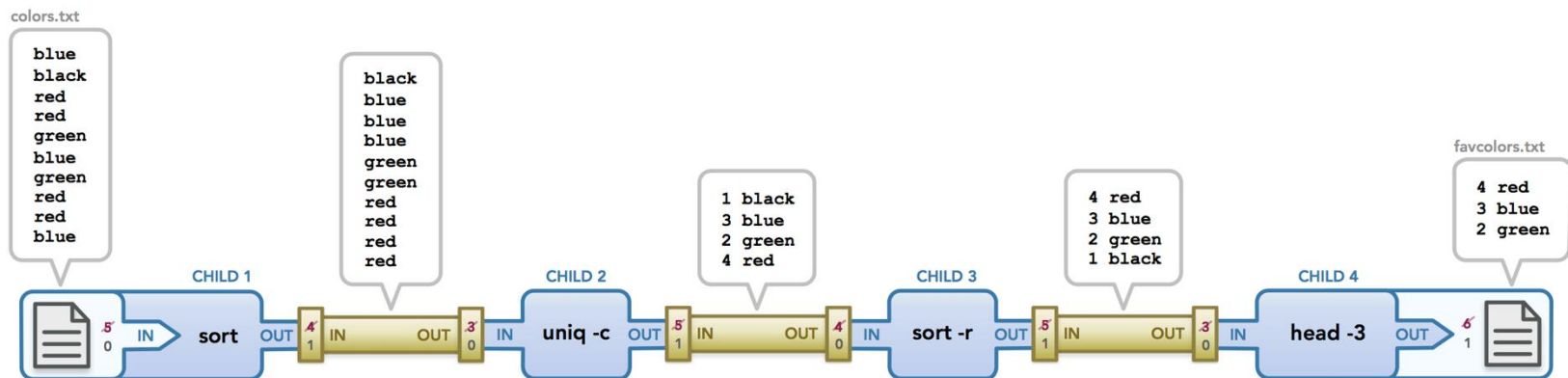
- **Recall that writing or reading from a pipe is an atomic operation, if the data size is less than `PIPE_BUF`**

Pipes and exec functions

- **Exec functions allow a process to run external code within the same process**
 - File descriptors remain open across `exec` unless explicitly changed via `fcntl()`
- **The new program code cannot access any previous variables**
 - Including arrays holding pipe descriptors, because the original process image is overwritten
- **How can we design multiple processes, each running an external executable, so they run concurrently and exchange data through pipes?**

Pipes and exec functions

- Most programs are designed to read data from `stdin` and output data to `stdout`



- Redirecting the flow of bytes from/to a pipe requires **manipulating the descriptor table**
 - Multiple descriptors may refer to the same resource

The dup function

```
#include <unistd.h>

int dup(int fd);
```

■ Duplicates the descriptor in `fd`

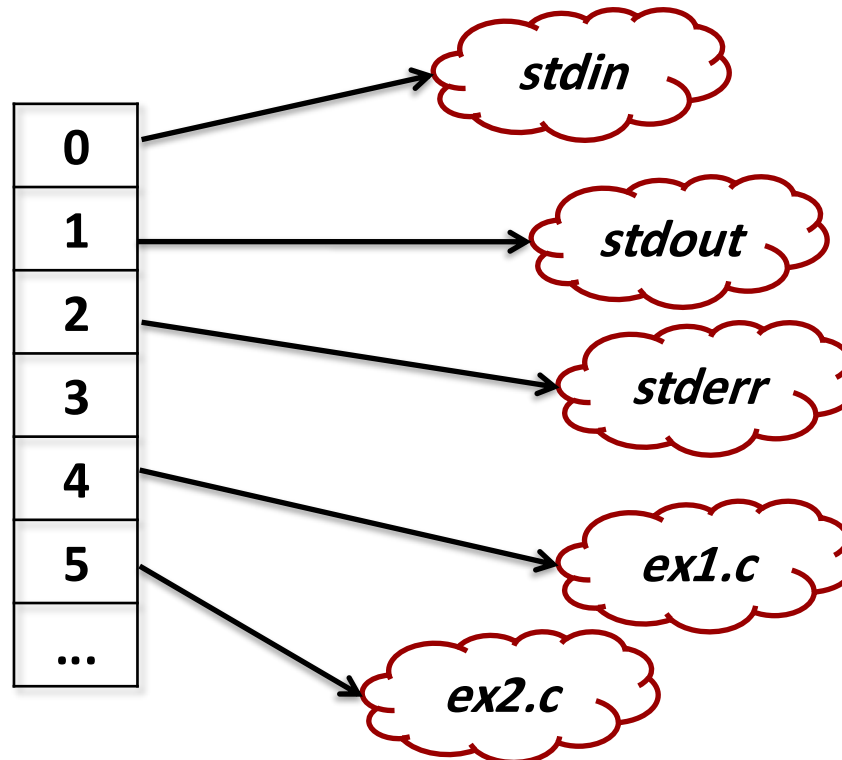
- POSIX requires that any function that allocates a new file descriptor must always choose the **lowest-numbered unused descriptor** in that process

■ Returns

- The new descriptor (a different number) that points to the same file/resource
- -1, if a failure occurs

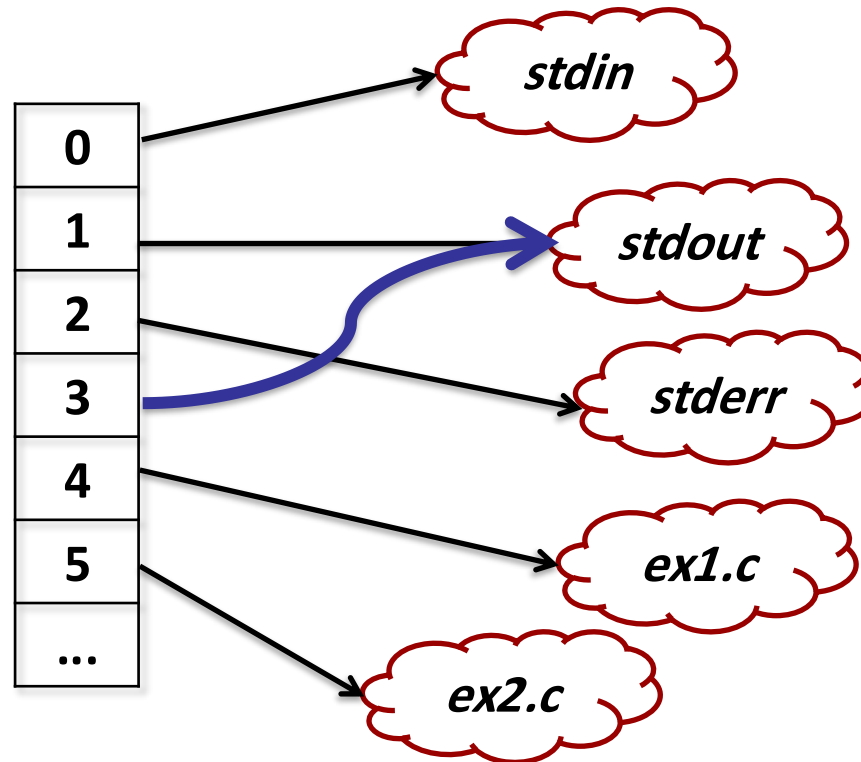
Example – dup ()

- Assuming the process's descriptor table as:



Example – dup ()

■ `dup (1) ;`



The dup2 function

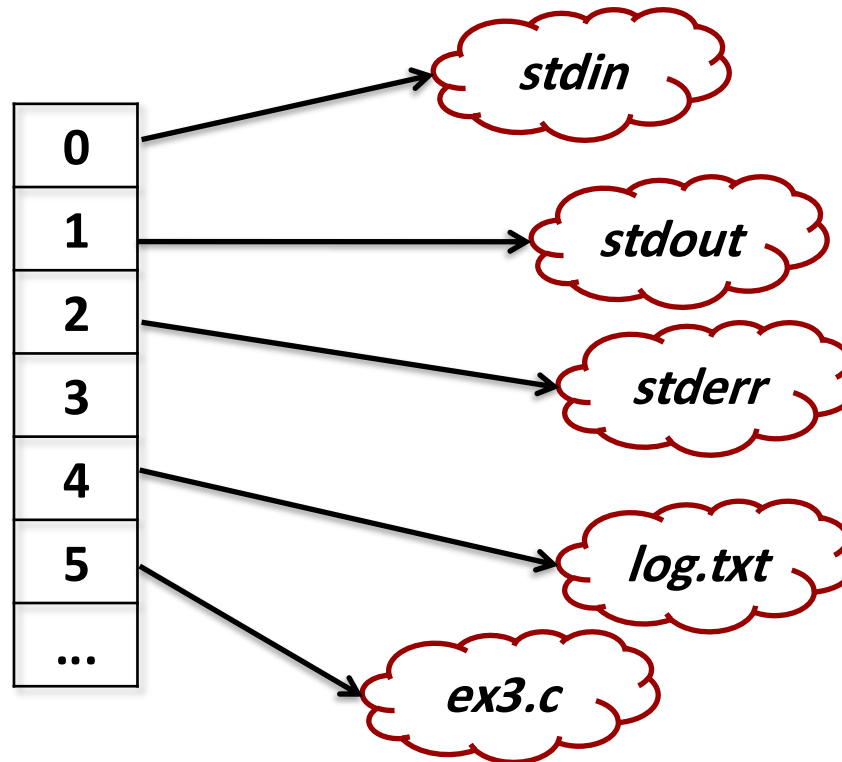
```
#include <unistd.h>

int dup2(int fd1, int fd2);
```

- **Duplicates fd1 to the position of fd2**
 - (instead of a free position)
- **If fd2 was previously open, it is closed before being reused**
 - Closing and reusing fd2 is **atomic**
- **Returns**
 - A value that is equal to the value of fd2, **if** successful
 - -1, if a failure occurs, it returns a value of -1

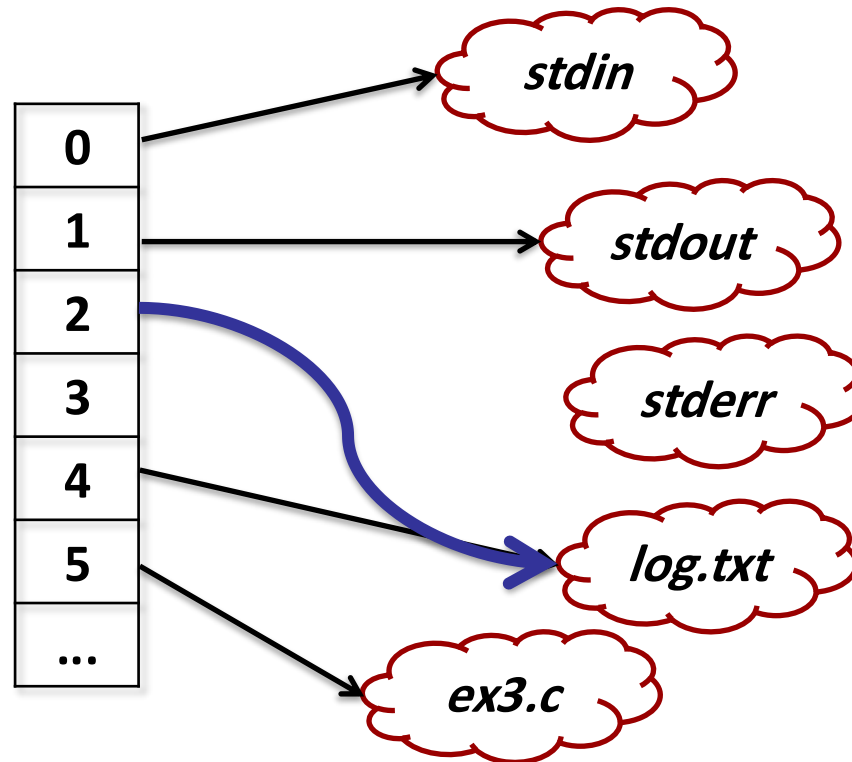
Example – dup2 ()

- Assuming the process's descriptor table as:



Example – dup2 ()

■ `dup2 (4 , 2) ;`



Example – redirect stdout

- Consider the following code compiled to executable `test1`

```
int main(){  
  
    int i;  
  
    for(i=0;i<5;i++){  
        fprintf(stdout, "Line %d\n", i+1);  
        sleep(1);  
    }  
  
    return 0;  
}
```

- Implement a program that creates a child process to execute `test1` and send its output to the parent through a pipe

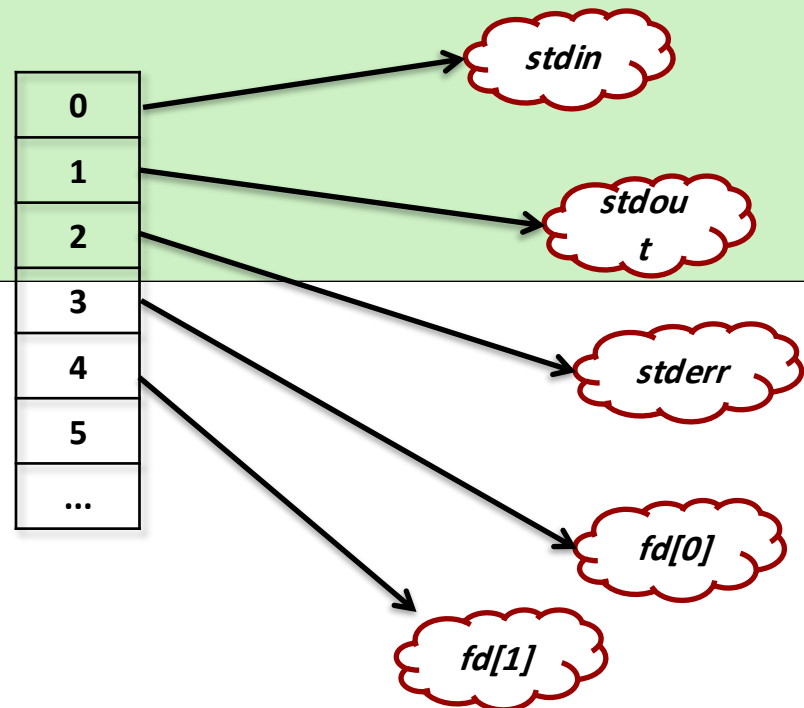
Example – redirect stdout

```
#define BUFFER_SIZE 80

int main(void){
    char buffer[BUFFER_SIZE];
    int fd[2],i,n;
    pid_t pid;

    /* creates a pipe */
    if(pipe(fd) == -1){
        perror("Pipe failed");
        exit(1);
    }
}
```

Parent file descriptors



Example – redirect stdout

```

/* child inherits file descriptor table */
pid = fork();

if(pid == 0){
    /* prints to stdout */
    printf("Child process\n");

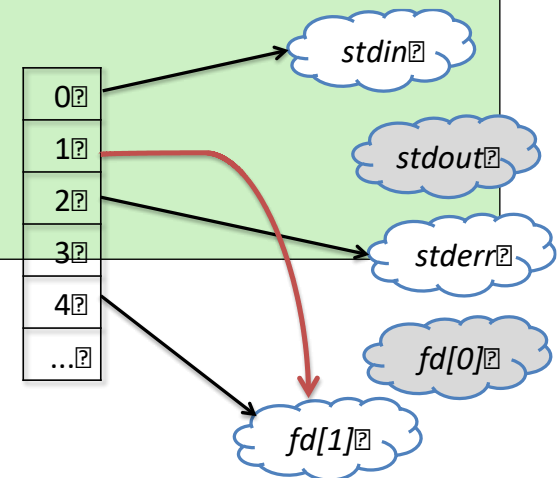
    /* closes unused descriptors and redirects stdout to pipe */
    close(fd[0]);
    dup2(fd[1], 1);
    close(fd[1]);

    
    execvp("./test1", "./test1", (char*)NULL);

    perror("exec error");
    exit(1);
}

```

Child file descriptors



Example – redirect stdout

```
else{
    printf("Parent...\n");

    /* closes unused write descriptor */
    close(fd[1]);

    /* reads from pipe (production code must check for errors) */
    while((n=read(fd[0],buffer,BUFFER_SIZE-1))!=0){
        buffer[n] = 0;
        printf("Parent read: %s\n",buffer);
    }

    /* closes read descriptor */
    close(fd[0]);
    wait(NULL);
}
```


Exercise 2

- **Implement a program that creates two child processes**
 - Child 1 executes the command `cat fx.txt`, sending the output to child 2 using a pipe
 - Child 2 executes `sort`, receiving its input from the pipe and sending the output to `stdout`
- **The parent waits for both children to terminate**